

# Computer Organization & Assembly Language (EE2003)

Date: September  
22<sup>nd</sup> 2026

## Course Instructor(s)

Dr. Atira Anum, Ms. Section  
Samin Iftikhar, Ms.  
Aleena Ahmad, Ms.  
Sana Fatima, Mr.  
Salman Mubarak,  
Ms. Saira Arif, Ms.  
Shazia Ahmed

Do not write below this line

## Instruction/Notes:

1. This is an open book exam. Only textbook by “Belal Hashmi” is allowed. Sharing book or calculators is NOT

Student Signature

**ALLOWED. PAPERS/NOTES ATTACHED WITH BOOK ARE NOT ALLOWED.**

2. Answer Q1 and Q2 on the question paper and Q3 on the answer sheet.

3. In case of any ambiguity, make reasonable assumptions. Questions during exams are not allowed.

***CLO # 1: Demonstrate the basic concepts of computer organization including CPU, memories, and input/output and explain their purposes and interactions.***

## Q1: Short Questions [1+ 4 + 5 marks]

(a) How many address bits do we need to access 64GB memory? 36.

(b) For each of the instructions given below, identify whether the instruction is valid or invalid.

## Sessional-I Exam

|                          |                      |                    |                      |
|--------------------------|----------------------|--------------------|----------------------|
| <b>Total Time (Hrs):</b> | <b>1</b>             |                    |                      |
| <b>Total Marks:</b>      | <b>45</b>            |                    |                      |
| <b>Total Questions:</b>  | <b>3</b>             |                    |                      |
| <b>Instruction</b>       | <b>Valid/Invalid</b> | <b>Instruction</b> | <b>Valid/Invalid</b> |
| mov [bx-si], 2           | Invalid              | mov [bp+bx+si], 2  | Invalid              |
| mov [0x1234], 2          | Invalid              | mov bl, [bx]       | Valid                |

(c) Study the following listing file and answer the questions given below:

|                            |                                |
|----------------------------|--------------------------------|
| 0400 B80010 mov ax, 0x1000 | <b>Show your working here.</b> |
|----------------------------|--------------------------------|

# National University of Computer and Emerging Sciences

## Lahore Campus

|      |        |                |
|------|--------|----------------|
| 0403 | BB3412 | mov bx, 0x1234 |
| 0406 | 01D8   | add ax, bx     |
| 0408 | 89C1   | mov cx, ax     |
| 040A | 29D8   | sub ax, bx     |
| 040C | 05FF00 | add ax, 0x00FF |
| 040F | 40     | inc ax         |

1. What is the offset address of the instruction add ax, 0x00FF? 0x040C
2. Determine the machine-code bytes for the instruction at offset 0x040A. 29D8
3. What are the final values of ax and cx? ax: 0x1100, cx: 0x2234
4. What is the total length, in bytes, of the complete instruction sequence? 16 bytes

**CLO # 2: Describe the working of important x86 assembly primitives, including arithmetic, branching, bit manipulation, addressing modes and interrupt handling.**

**Q2: [5+5+3+3+4 marks]**

- (a) Consider the following instruction.

**mov ax, [bx+si+0x20]**

Given: DS = 0x1800, BX = 0x2400, SI = 0x0150

1. Identify the addressing mode used in the above instruction. Base + Index + Offset
2. Calculate the physical address of the memory location read by the above instruction.

**Show your working here.**

**Effective Address = 0x2400 + 0x0150 + 0x20 = 0x2570**

**Physical Address = 18000 + 2570 = 0x1A570**

- (b) Two segment registers are loaded with DS = 0x1A20 and ES = 0x1B00.
- a. Determine whether Segment 1 DS and Segment 2 ES overlap.
  - b. If they overlap, calculate the exact number of overlapping bytes (in decimal and hex).

# National University of Computer and Emerging Sciences

## Lahore Campus

**Show your working here.**

**1. Calculate Start and End Addresses:**

○ DS Range:

Start\_DS = 1A200h

End\_DS = 1A200h + FFFFh = 2A1FFh

○ ES Range:

Start\_ES = 1B000h

End\_ES = 1B000h + FFFFh = 2AFFFh

**2. Check Overlap Condition: Since Start\_ES (1B000 h) <= End\_DS (2A1FF h), the two segments overlap.**

**3. Calculate Overlapping Bytes: Distance = 1B000 h - 1A200 h = 00E00 h**

Overlapping Bytes (h) = 10000 h - 00E00 h = F200 h

Overlapping Bytes (Decimal) = 61,952 bytes

(c) Answer following questions for the code snippet given below.

```
[org 0x0100]
mov al, 5
cmp al, -3
ja skip
mov bl, 1
jmp terminate
skip:
mov bl, 2
terminate:
mov ax, 0x4C00
int 0x21
```

**Show your working here.**

```
0000 0101
- 1111 1101
= 0000 1000
```

- What will be the value of BL after execution of this program? **BL:** 1
- What will be the values of the following flags after execution of **cmp** instruction?

**OF:** 0, **SF:** 0, **ZF:** 0, **CF:** 1

(d) Write the appropriate logical instruction to perform the following operations.

- Clear bits 7–4 while preserving bits 3–0. **Instruction:** AND AL, 0FH
- Set bit 6 without changing any other bit. **Instruction:** OR AL, 40H
- Invert bits 2 and 0 using a single instruction. **Instruction:** XOR AL, 05H

(e) What would be the values of AH and AL after execution of the following instructions. The initial value of AX is 0x1F2F. Assume each instruction is independent and write AH and AL values in Hexadecimal.

| Instruction | AL   | AH   |
|-------------|------|------|
| ror ax, 8   | 0x1F | 0x2F |
| sar ax, 1   | 0x97 | 0x0F |

# National University of Computer and Emerging Sciences

## Lahore Campus

***CLO # 3: Apply the knowledge of Intel x86 architecture to develop moderately complex and well-modularized assembly programs.***

---

**Q3: [15 marks]** Given a word array **array1** and a target sum **k**, write a complete assembly code to find the count of all pairs of elements whose sum equals **k**. Store the count in the data label named **"count"**.

For example, the given array is **{1, 5, 7, -1, 5}** and the target sum is **6**, then the pairs having sum equal to 6 are (1, 5) and (7, -1) and a second (1, 5) so the **count** will be equal to 3.

Your program should be generic and works for any array and any number **k**. Assume that the array size is given in a data label named **'size'**.

You can define your data labels as follows:

```
array1: dw 1, 5, ,7, -1, 5
size: dw 5
k: dw 6
count: dw 0
```